

```

xSpeed = N_PIXELS_PER_FRAME
ySpeed = N_PIXELS_PER_FRAME

# 6 - Loop forever
while True:

    # 7 - Check for and handle events
    for event in pygame.event.get():
        # Clicked the close button? Quit pygame and end the program
        if event.type == pygame.QUIT:
            pygame.quit()
            sys.exit()

    # 8 - Do any "per frame" actions
    ❷ if (ballRect.left < 0) or (ballRect.right >= WINDOW_WIDTH):
        xSpeed = -xSpeed # reverse X direction

    if (ballRect.top < 0) or (ballRect.bottom >= WINDOW_HEIGHT):
        ySpeed = -ySpeed # reverse Y direction

    # Update the ball's rectangle using the speed in two directions
    ballRect.left = ballRect.left + xSpeed
    ballRect.top = ballRect.top + ySpeed

    # 9 - Clear the window before drawing it again
    window.fill(BLACK)

    # 10 - Draw the window elements
    ❸ window.blit(ballImage, ballRect)

    # 11 - Update the window
    pygame.display.update()

    # 12 - Slow things down a bit
    clock.tick(FRAMES_PER_SECOND)

```

***Listing 5-7:** A location-based animation, bouncing a ball around the window, using `rects`*

This approach of using a `rect` object is neither better nor worse than using separate variables. The resulting program works exactly the same as the original. The important lesson here is how you can use and manipulate attributes of a `rect` object.